



INSTITUT  
POLYTECHNIQUE  
DE PARIS

# Module 2 : Principes SOLID & Qualité Logicielle

Python & Java

S.O.L.I.D

Clean Code

Qualité · Refactoring

MODULE 2 · Ingénierie Logicielle Évaluation : 40% TP · 60% Contrôle Continu

## La Roadmap du Coursus : Ingénierie Logicielle

**01**

**Modélisation  
Objet Avancée**

Classes, Héritage,  
Encapsulation, Polymorphisme

**02**

**Principe SOLID  
& Qualité**

5 Principes SOLID,  
Clean Code, Refactoring

**03**

**Patrons de  
Conception**

Design Patterns GoF  
(Création, Structure, Compt.)

**04**

**Modélisation  
UML**

Représentation visuelle  
des architectures

Objectif final : Concevoir · Implémenter · Tester · Livrer un logiciel de qualité industrielle

## Pourquoi SOLID ? Le Code Spaghetti

### Sans SOLID, le code devient...

- Difficile à lire  
Modifications impossibles sans tout casser
- Impossible à tester  
Chaque classe dépend de tout
- Fragile  
Corriger un bug en crée trois nouveaux
- Non-réutilisable  
Code copié-collé partout

## S O L I D

Single Responsibility

Open / Closed

Liskov Substitution

Interface Segregation

Dependency Inversion

**"SOLID = le code que tu seras fier de relire dans 6 mois, et que ton collègue comprendra sans toi."**

# Principe S — Single Responsibility

"Une classe = une seule raison de changer"

## 01 God Class



**La classe fait tout : donnée**

La classe fait tout : données, calcul, affichage, persistance.

## 02 Découper



**Chaque classe a une seule responsabilité**

Chaque classe a une seule responsabilité bien définie.

## 03 Résultat



**Code lisible, testable, modifiable**

Code lisible, testable, facile à modifier sans casser le reste.

# SRP · Single Responsibility Principale · Bad vs Good

## S — SRP : Violation vs Principe Appliqué · CompteBancaire

Problème : 1 classe qui fait TOUT. Solution : 4 classes spécialisées, chacune avec 1 seule raison de changer.

### Python · God Class (MAUVAIS)

MAUVAIS

```
class CompteBancaire:
    def __init__(self, t, s):
        self.titulaire = t
        self.solde = s
    def retirer(self, m):
        self.solde -= m # logique
    def afficher_solde(self):
        print(f'Solde: {self.solde}') # affichage
    def sauvegarder_bdd(self):
        db.save(self.titulaire, self.solde) # BDD
    def envoyer_email(self):
        mail.send('alerte@banque.fr') # email
    def valider_iban(self, iban):
        return len(iban) == 27 # validation
```

### Python · 4 Classes SRP (BON)

BON

```
class CompteBancaire: # donnees + logique
    def __init__(self, t, s):
        self.__titulaire = t
        self.__solde = s
    def retirer(self, m):
        if m > 0: self.__solde -= m

class AffichageCompte: # affichage seul
    def afficher(self, c):
        print(f'Solde: {c.get_solde()}')

class RepoCompte: # persistance seule
    def sauvegarder(self, c):
        db.save(c)

class NotifCompte: # email seul
    def notifier(self, msg): mail.send(msg)
```

# Principe O — Open / Closed

"Ouvert a l'extension, Ferme a la modification"

## 01 Problème



**if/elif infini : ajouter un**

if/elif infini : ajouter un type = modifier le code existant.  
Risque de régression.

## 02 Principe



**Étendre par interface ou héritage**

Étendre par interface ou héritage sans toucher au code  
déjà testé.

## 03 Bénéfice



**Zéro modification des classe**

Zéro modification des classes existantes. Nouveau type  
= nouvelle classe.

## O — OCP : if/elif (MAUVAIS) vs Interface Strategy (BON)

### Python · if/elif (MAUVAIS)

MAUVAIS

```
class CompteBancaire:
    def calculer_frais(self, type_compte):
        # MAUVAIS : if/elif infini
        if type_compte == 'cheque':
            return 2.0
        elif type_compte == 'epargne':
            return 0.5
        elif type_compte == 'premium':
            return 0.0
        # Ajouter 'pro' ? Modifier ICI !
        # Risque de casser cheque et epargne
```

### Java · switch (MAUVAIS)

MAUVAIS

```
public class CompteBancaire {
    double calculerFrais(String type) {
        // MAUVAIS : switch infini
        switch(type) {
            case "cheque": return 2.0;
            case "epargne": return 0.5;
            // Ajouter "pro" ?
            // MODIFIER ICI = risque régression
        }
        return 0;
    }
}
```

### Python · Strategy OCP (BON)

BON

```
# OCP : Strategy Pattern
class IFrais: # abstraction
    def calculer(self): pass

class FraisCheque(IFrais):
    def calculer(self): return 2.0

class FraisEpargne(IFrais):
    def calculer(self): return 0.5

class FraisPro(IFrais): # NOUVEAU type
    def calculer(self): return 1.5
# Aucune ligne existante modifiée !
```

### Java · Interface (BON)

BON

```
// OCP : interface + implémentations
interface IFrais {
    double calculer();
}
class FraisCheque implements IFrais {
    public double calculer(){ return 2.0; }
}
class FraisEpargne implements IFrais {
    public double calculer(){ return 0.5; }
}
// NOUVEAU - zéro modif existante
class FraisPro implements IFrais {
    public double calculer(){ return 1.5; }
}
```

# Principe L — Liskov Substitution

"Un enfant doit remplacer son parent sans surprise"

## 01 Exemple



**CompteEpargne herite de CompteBancaire**

CompteEpargne herite de CompteBancaire : retirer() doit marcher pareil.

## 02 Violation



**CompteEpargne.retirer()**

CompteEpargne.retirer() lance une exception que le parent ne lancait jamais.

## 03 Règle



**Même contrat : même valeur de retour**

Même contrat : même valeur de retour, même gestion d'erreur.

## L — LSP : le sous-type doit honorer le contrat de la classe parente

Si `CompteEpargne` est substituable à `CompteBancaire`, tout code qui utilise `CompteBancaire` doit fonctionner avec `CompteEpargne`.

### Python · LSP Violé (MAUVAIS)

MAUVAIS

```
class CompteBancaire:
    def retirer(self, montant):
        if montant <= self.solde:
            self.solde -= montant
        # Pas d'exception jamais

class CompteEpargne(CompteBancaire):
    def retirer(self, montant):
        # MAUVAIS : change le contrat !
        if montant > 500:
            raise Exception('Plafond !') # NOUVEAU comportement
        # Le parent ne lance jamais d'exception
        # LSP VIOLÉ : substitution impossible
```

### Python · LSP Respecté (BON)

BON

```
class CompteBancaire:
    def retirer(self, montant) -> bool:
        # Contrat : retourne True/False
        if montant <= self.solde:
            self.solde -= montant
            return True
        return False # jamais d'exception

class CompteEpargne(CompteBancaire):
    def retirer(self, montant) -> bool:
        # Respecte EXACTEMENT le contrat
        if montant > 500: return False
        if montant > self.solde: return False
        return super().retirer(montant)
```

**Test de substitution : remplace `CompteBancaire` par `CompteEpargne` dans ton `main()`. Tout doit fonctionner sans modification.**

# Principe I — Interface Segregation

"Ne force pas une classe a implementer ce dont elle n'a pas besoin"

## 01 Problème



**Une grosse interface force C**

Une grosse interface force CompteEpargne a implementer decouvert() inutile.

## 02 Solution



**Découper : IRetrait, IVirement**

Découper : IRetrait, IVirement, IDecouvert. Chaque classe prend ce qu'elle utilise.

## 03 Signal



**Si tu vois NotImplementedError**

Si tu vois NotImplementedError dans une méthode : violation ISP détectée.

## I — ISP : une interface = un rôle. Ne pas forcer l'inutile.

Signal ISP violé : une méthode qui contient NotImplementedError ou pass uniquement = la classe n'en a pas besoin.

### Python · Interface grosse (MAUVAIS)

MAUVAIS

```
# MAUVAIS : interface trop grosse
class ICompte: # tout en une
    def retirer(self, m): pass
    def virer(self, dest, m): pass
    def autoriser_decouvert(self): pass
    def placer_epargne(self): pass

class CompteEpargne(ICompte):
    def retirer(self, m): ...
    def virer(self, dest, m): ...
    def autoriser_decouvert(self):
        raise NotImplementedError() # ISP violé !
    def placer_epargne(self): ...
```

### Java · Interfaces séparées (BON)

BON

```
// BON : interfaces spécialisées
interface IRetrait {
    void retirer(double m);
}

interface IDecouvert {
    void autoriserDecouvert(double max);
}

// CompteEpargne : seulement IRetrait
class CompteEpargne implements IRetrait {
    public void retirer(double m) { ... }
    // Pas de decouvert = pas d'ISP
}

// CompteCheque : les deux
class CompteCheque implements
    IRetrait, IDecouvert { ... }
```

**Règle : si une méthode d'interface ne sert pas à une classe, l'interface est trop grosse. Découper immédiatement.**

# Principe D — Dependency Inversion

"Dependre d'abstractions, jamais de classes concrètes"

## 01 Problème



### CompteBancaire crée MySQL directement

CompteBancaire crée MySQL directement. Changer de BDD = modifier la logique.

## 02 Principe



### Injecter IRepo dans le constructeur

Injecter IRepo dans le constructeur. CompteBancaire ne sait pas quelle BDD.

## 03 Bénéfice



### Tester avec RepoMock. Change

Tester avec RepoMock. Changer MySQL par PostgreSQL : zéro ligne de logique modifiée.

## D — DIP : dépendre d'abstractions, pas d'implémentations concrètes

Bénéfice clé : injecter RepoMock en test → tester la logique de retrait sans aucune base de données réelle.

### Python · new MySQL() dur (MAUVAIS)

MAUVAIS

```
# MAUVAIS : dépendance concrète
class CompteBancaire:
    def __init__(self, titulaire, solde):
        self.titulaire = titulaire
        self.solde = solde
        # Dépend directement de MySQL !
        self.repo = RepoBDD_MySQL()

    def sauvegarder(self):
        self.repo.save(self)
        # Changer BDD ? Modifier ici.
        # Tester sans BDD réelle ? Impossible.
        # DIP VIOLÉ : couplage fort
```

### Python · Injection IRepo (BON)

BON

```
# BON : injection de dépendance
class IRepo: # abstraction
    def save(self, compte): pass

class RepoBDD(IRepo):
    def save(self, compte): ...

class RepoMock(IRepo): # tests
    def save(self, compte): pass

class CompteBancaire:
    def __init__(self, t, s, repo: IRepo):
        self.__repo = repo # injecté !
    def sauvegarder(self):
        self.__repo.save(self) # abstraction
```

## La God Class : quand une classe viole les 5 principes SOLID en même temps

### S

Violation  
Données + logique  
+ affichage + BDD  
= 6 responsabilités

### O

Violation  
if/elif sur type  
Ajouter un type =  
modifier la classe

### L

Violation  
Enfants changent  
le contrat de  
retirer()

### I

Violation  
Interface unique  
avec méthodes  
inutilisées

### D

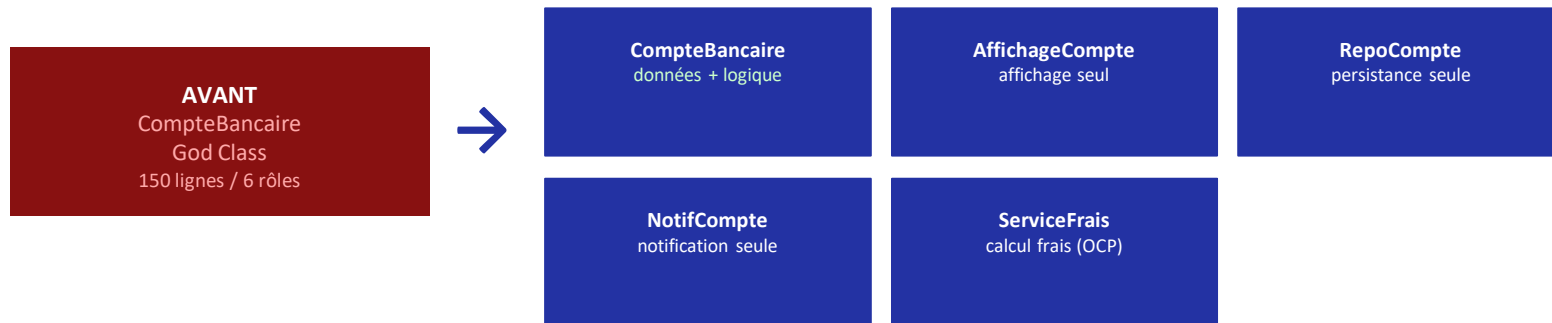
Violation  
MySQL, SMTP  
codés en dur  
= couplage fort

CompteBancaire God Class · Python (viole les 5 SOLID)

MAUVAIS

```
class CompteBancaire: # 1 classe = TOUT faire
    def __init__(self, t, s):
        self.t=t; self.s=s
        self.db = MySQL() # DIP violé : dépend concret
        self.mail = SMTP() # DIP violé
    def retirer(self, m): self.s -= m # logique ok
    def afficher(self): print(self.s) # SRP violé : affichage
    def sauvegarder(self): self.db.save(self) # SRP + DIP violé
    def notifier(self): self.mail.send('alerte') # SRP violé
    def calculer_frais(self, type_compte): # OCP violé : if/elif
        if type_compte=='cheque': return 2.0
        elif type_compte=='epargne': return 0.5 # Ajouter = modifier ici
```

## De la God Class a 5 Classes SOLID — Le Refactoring Complet



### Python · Architecture SOLID Finale (BON — respecte S, O, L, I, D)

BON

```
class CompteBancaire: # SRP: données + logique seulement
    def __init__(self, titulaire, solde, repo: IRepo): # DIP: IRepo injecté
        self.__titulaire = titulaire # encapsulation M1
        self.__solde = solde
        self.__repo = repo # abstraction, pas MySQL
    def retirer(self, montant) -> bool: # LSP: contrat clair bool
        if montant > 0 and montant <= self.__solde:
            self.__solde -= montant; return True
        return False # jamais d'exception: LSP respecté
class ServiceFrais: # OCP: ajouter stratégie sans modifier
    def calculer(self, strategie: IFrais): return strategie.calculer()
```

### Python vs Java : Application des 5 Principes SOLID

| Principe | Python                                            | Java                                              |
|----------|---------------------------------------------------|---------------------------------------------------|
| S — SRP  | 1 module .py = 1 classe. Convention standard.     | 1 fichier .java = 1 public class.                 |
| O — OCP  | Héritage + duck typing. Pas d'interface formelle. | interface + implements. Vérifié compilation.      |
| L — LSP  | Type hints PEP484 recommandés. Vérif runtime.     | Typage statique + @Override explicite.            |
| I — ISP  | ABC / Protocol Python 3.8+. Léger en syntaxe.     | Interfaces Java natives. default methods Java 8+. |
| D — DIP  | Injection constructeur. FastAPI DI intégré.       | Spring @Autowired / @Inject. Natif Java EE.       |

### Synthèse : Pourquoi SOLID transforme la qualité de ton code

#### Maintenabilité via SRP

Chaque classe a 1 seule raison de changer.  
Un bug isolé = fix local, sans effet de bord.

#### Extensibilité via OCP

Ajouter un type de compte sans  
toucher au code existant.

#### Fiabilité via LSP

Substitution garantie :  
les sous-classes honorent le contrat.

#### Testabilité via DIP

Injection de dépendance :  
tester avec mocks sans BDD réelle.

**Fil rouge : CompteBancaire M1 → refactoré SOLID M2 → enrichi de Patterns M3 → modélisé UML M4**

### Mindset Ingénieur : 5 Réflexes pour Détecter les Violations SOLID

**S** Je vois une méthode qui fait calcul ET affichage ET BDD ? → Violation SRP → Découper en classes spécialisées.

**O** Je vois un if/elif ou switch sur le type d'objet ? → Violation OCP → Remplacer par interface + polymorphisme.

**L** Une méthode enfant lève une exception que le parent ne lève pas ? → Violation LSP → Revoir le contrat.

**I** Une méthode contient NotImplementedError / pass inutile ? → Violation ISP → Découper l'interface.

**D** Un new ConcreteClass() dans un constructeur ? → Violation DIP → Injecter via interface dans le constructeur.

**Avant d'écrire une ligne : quelle est l'unique responsabilité de cette classe ? Si tu hésites : violation SRP.**

# SOLID : Tableau Récapitulatif Complet

| Principe | Définition                    | Signal violation               | Python                   | Java                       |
|----------|-------------------------------|--------------------------------|--------------------------|----------------------------|
| <b>S</b> | 1 classe = 1 responsabilité   | God Class, méthode fait tout   | 1 module = 1 classe      | 1 fichier .java = 1 classe |
| <b>O</b> | Ouvert extension, fermé modif | if/elif sur type d'objet       | Héritage + duck typing   | interface + implements     |
| <b>L</b> | Enfant remplace parent        | Exception inattendue en enfant | Type hints + bool return | @Override + même contrat   |
| <b>I</b> | Petites interfaces précises   | NotImplementedError partout    | ABC / Protocol 3.8+      | Interfaces Java séparées   |
| <b>D</b> | Dépendre d'abstractions       | new ConcreteClass() dur        | Injection constructeur   | Spring @Autowired          |

## TP 2 — Dans la pratique du TP1 HEP Gestion des étudiants appliqué les 5 Principes SOLID en 2h

1

SRP

2

OCP

3

LSP

4

ISP

5

DIP

Durée estimée : SRP 45min · OCP 30min · LSP 15min · DIP 20min ·

## Ressources Complémentaires · Module 2 SOLID

### Ressources Python

koor.fr — tutoriels Python OOP  
PEP8 — style & conventions  
Python ABC docs.python.org  
pytest — tests unitaires

### Ressources Java

koor.fr — tutoriels Java OOP  
docs.oracle.com — Java SE 17  
JUnit 5 — junit.org  
Spring SOLID — spring.io

### Outils IDE recommandés

VS Code + Python Extension  
IntelliJ IDEA CE (Java)  
PyCharm CE (Python)  
Draw.io — diagrammes classes

### Prochaine étape : Module 3

Design Patterns GoF  
Strategy + Observer sur CompteBancaire  
Factory Method + Singleton  
Lien avec OCP et DIP (Module 2)

🔗 <https://refactoring.guru/design-patterns> · Tutoriels, supports de cours, exemples de codes et quiz de validation Java & Python

# Fin du Module 2

Place à la Pratique SOLID !

Travaux Pratiques (TP) : Python & Java

Objectif : Refactorer un use case avec les 5 principes SOLID · 2h

**01**

**SRP**

God Class  
→ 4 classes

**02**

**OCP**

if/elif  
→ interface

**03**

**LSP**

Contrat  
retirer()

**04**

**ISP**

Violation

**05**

**DIP**

Injection  
IRepo